Jr.Pen 7.4. Intro to SSRF

Task 1 - What is an SSRF?

SSRF stands for Server-Side Request Forgery. It's a vulnerability that allows a malicious user to cause the webserver to make an additional or edited HTTP request to the resource of the attacker's choosing. There are two types of SSRF vulnerability; the first is a regular SSRF where data is returned to the attacker's screen. The second is a Blind SSRF vulnerability where an SSRF occurs, but no information is returned to the attacker's screen. A successful SSRF attack can result in any of the following:

Access to unauthorised areas.
Access to customer/organisational data.
Ability to Scale to internal networks.
Reveal authentication tokens/credentials.

> What does SSRF stand for? Server-Side Request Forgery

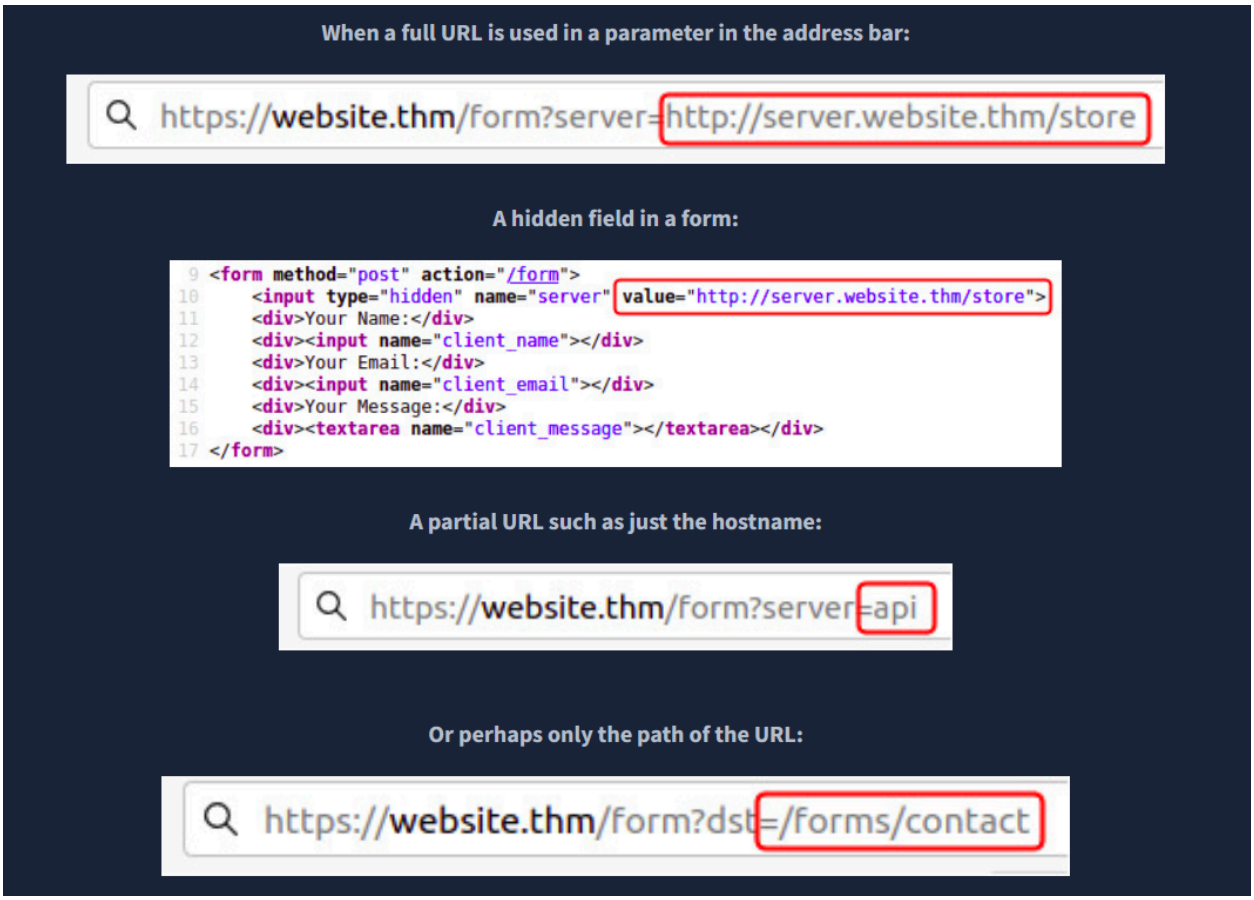
> As opposed to a regular SSRF, what is the other type? Blind

Task 2 - SSRF Examples

Start by clicking on the visit site button. The example shows how an attacker can alter a request and force the application to send a request of the attacker’s choosing. In the end Change this URL https://website.thm/item/2?server=api to https://website.thm/item/2?server=server.website.thm/flag?id=9&x=

Note: the &x= at the end of a URL which is simply a query parameter with an empty value.

Task 3 - Finding an SSRF



> Based on simple observation, which of the following URLs is more likely to be vulnerable to SSRF?

https://website.thm/index.phphttps://website.thm/list-products.php?categoryId=5325https://website.thm/fetch-file.php?fname=242533.pdf&srv=filestorage.cloud.thm&port=8001https://website.thm/buy-item.php?itemId=213&price=100&q=2

A. 3
Explanation: Potential SSRF vulnerabilities can be found wherever a web application accepts user input that is later used to make a server-side request. Common places include parameters that accept a full URL, hidden form fields, inputs that allow only a hostname, or fields that accept just the URL path. Some of these are easy to exploit, while others require careful testing and trial and error to discover a successful SSRF payload. Based on simple observation, the URL most likely to be vulnerable to SSRF is: Option 3

Why this one stands out:

It contains parameters like srv (server/host) and port
This strongly suggests the backend is connecting to another server to fetch a file
User-controlled host and port values are classic SSRF indicators
Why the others are less likely

index.php → no parameters, nothing to manipulate
list-products.php?categoryId=5325 → numeric ID, typical database lookup
buy-item.php?itemId=213&price=100&q=2 → business logic parameters, not network-related

Task 4 - Defeating Common SSRF Defenses

There are usually two approaches to this, either a deny list or an allow list.

Deny List: A Deny List is where all requests are accepted apart from resources specified in a list or matching a particular pattern. A Web Application may employ a deny list to protect sensitive endpoints, IP addresses or domains from being accessed by the public while still allowing access to other locations. Also, in a cloud environment, it would be beneficial to block access to the IP address 169.254.169.254, which contains metadata for the deployed cloud server, including possibly sensitive information. An attacker can bypass this by registering a subdomain on their own domain with a DNS record that points to the IP Address 169.254.169.254.

- **Common Deny List Targets (Blocked Destinations)**
  - localhost
  - 127.0.0.1
  - 0.0.0.0
  - ::1 (IPv6 localhost)
- **Private / Internal IP Ranges**
  - 10.0.0.0/8
  - 172.16.0.0/12
  - 192.168.0.0/16
- **Cloud Metadata Services**

    - 169.254.169.254 (AWS, Azure, GCP)
    - metadata.google.internal
- **Internal Hostnames**
    - internal
    - intranet
    - admin
    - private
- **Dangerous Protocols**
    - file://
    - gopher://
    - ftp://
    - dict://

Allow List: An allow list is where all requests get denied unless they appear on a list or match a particular pattern, such as a rule that an URL used in a parameter must begin with https://website.thm. An attacker could quickly circumvent this rule by creating a subdomain on an attacker's domain name, such as https://website.thm.attackers-domain.thm. The application logic would now allow this input and let an attacker control the internal HTTP request.

Fixed Trusted Domains
https://api.company.comhttps://storage.company.comhttps://cdn.company.com

Domain Suffix Rules
*.company.com
*.trusted-service.com

Protocol Restrictions
Only http:// and https://
Often HTTPS only

Port Restrictions
80
443

Open Redirect: An open redirect is an endpoint that automatically forwards users to another URL based on a parameter. While often used for tracking or marketing, it can be abused to bypass strict SSRF protections. If an application only allows requests to its own domain, an attacker can supply a trusted URL that points to an open redirect, causing the server to follow the redirect and send the internal request to an attacker-controlled destination. Security-aware developers often try to reduce SSRF risk by validating the resources a server is allowed to access, usually through deny lists or allow lists. Allow list and deny list are access control mechanisms used to decide what is permitted or blocked.

> What method can be used to bypass strict rules? Open Redirect

> What IP address may contain sensitive data in a cloud environment?
> 169.254.169.254

> What type of list is used to permit only certain input? Allow List

> What type of list is used to stop certain input? Deny List

Task 5 - SSRF Practical

> What is the flag from the /private directory?
> THM{YOU_WORKED_OUT_THE_SSRF}

Process: Begin by visiting the 1st url (https://10-49-188-195.reverse-proxy.cell-prod-ap-south-1b.vm.tryhackme.com/) and create an account. We had created an account named Vicky Visit the 2nd URL (https://10-49-188-195.reverse-proxy.cell-prod-ap-south-1b.vm.tryhackme.com/customers/new-account-page) for selecting Avatar. Select an avatar and click update avatar button. Again select avatar and right click, select inspect. Try changing the value attribute to /private. You will get an error.Try again, this time set the value attribute to "x/../private". On updation, you will see Current Avatar, empty. Inspect the current avatar label, where you will see an base64 encoded string.



Decode this using base64decode website and you will get the flag.